



DATN Bvhoang 2020710 - Đồ án tốt nghiệp

Đồ án tốt nghiệp kỹ sư (Trường Đại học Bách khoa Hà Nội)



messages.pdf_cover_qr_code_label

ĐẠI HỌC BÁCH KHOA HÀ NỘI
TRƯỜNG ĐIỆN - ĐIỆN TỬ



ĐỒ ÁN TỐT NGHIỆP ĐẠI HỌC

Đề tài:

**Xây dựng robot tự hành di động cùng
với ROS sử dụng LiDAR 2D**

Sinh viên thực hiện:

BÙI VIỆT HOÀNG Điện tử 02 - K65 MSSV: 20203710

Giảng viên hướng dẫn: TS. PHẠM DOÃN TÍNH

HÀ NỘI, 02-2025

**ĐẠI HỌC BÁCH KHOA HÀ NỘI
TRƯỜNG ĐIỆN - ĐIỆN TỬ**



ĐỒ ÁN TỐT NGHIỆP ĐẠI HỌC

Đề tài:

**Xây dựng robot tự hành di động cùng
với ROS sử dụng LiDAR 2D**

Sinh viên thực hiện:

BÙI VIỆT HOÀNG Điện tử 02 - K65 MSSV: 20203710

Giảng viên hướng dẫn: **TS. PHẠM DOÃN TĨNH**

HÀ NỘI, 02-2025

ĐÁNH GIÁ ĐỒ ÁN TỐT NGHIỆP

(DÀNH CHO CÁN BỘ HƯỚNG DẪN)

Tên đề tài: **Xây dựng robot tự hành di động cùng với ROS sử dụng LiDAR 2D**

Họ tên sinh viên: **Bùi Việt Hoàng**

MSSV: **20203710**

Cán bộ hướng dẫn: **TS. Phạm Doãn Tĩnh**

STT	Tiêu chí (Điểm tối đa)	Hướng dẫn đánh giá tiêu chí	Điểm tiêu chí
1	Thái độ làm việc (2,5 điểm)	Nghiêm túc, tích cực và chủ động trong quá trình làm ĐATN Hoàn thành đầy đủ và đúng tiến độ các nội dung được GVHD giao	
2	Kỹ năng viết quyển ĐATN (2 điểm)	Trình bày đúng mẫu quy định, bố cục các chương logic và hợp lý: Bảng biểu, hình ảnh rõ ràng, có tiêu đề, được đánh số thứ tự và được giải thích hay đề cập đến trong đồ án, có căn lề, dấu cách sau dấu chấm, dấu phẩy, có mở đầu chương và kết luận chương, có liệt kê tài liệu tham khảo và có trích dẫn, v.v. Kỹ năng diễn đạt, phân tích, giải thích, lập luận: Cấu trúc câu rõ ràng, văn phong khoa học, lập luận logic và có cơ sở, thuật ngữ chuyên ngành phù hợp, v.v.	
3	Nội dung và kết quả đạt được (5 điểm)	Nêu rõ tính cấp thiết, ý nghĩa khoa học và thực tiễn của đề tài, các vấn đề và các giả thuyết, phạm vi ứng dụng của đề tài. Thực hiện đầy đủ quy trình nghiên cứu: Đặt vấn đề, mục tiêu đề ra, phương pháp nghiên cứu/ giải quyết vấn đề, kết quả đạt được, đánh giá và kết luận. Nội dung và kết quả được trình bày một cách logic và hợp lý, được phân tích và đánh giá thỏa đáng. Biện luận phân tích kết quả mô phỏng/ phần mềm/ thực nghiệm, so sánh kết quả đạt được với kết quả trước đó có liên quan. Chỉ rõ phù hợp giữa kết quả đạt được và mục tiêu ban đầu đề ra đồng thời cung cấp lập luận để đề xuất hướng giải quyết có thể thực hiện trong tương lai. Hàm lượng khoa học/ độ phức tạp cao, có tính mới/tính sáng tạo trong nội dung và kết quả đồ án.	
4	Điểm thành tích (1 điểm)	Có bài báo KH được đăng hoặc chấp nhận đăng/ đạt giải SV NCKH giải 3 cấp Trường trở lên/ Các giải thưởng khoa học trong nước, quốc tế từ giải 3 trở lên/ Có đăng ký bằng phát minh sáng chế. (1 điểm) Được báo cáo tại hội đồng cấp Trường trong hội nghị SV NCKH nhưng không đạt giải từ giải 3 trở lên/ Đạt giải khuyến khích trong cuộc thi khoa học trong nước, quốc tế/ Kết quả đồ án là sản phẩm ứng dụng có tính hoàn thiện cao, yêu cầu khối lượng thực hiện lớn. (0,5 điểm)	
Điểm tổng các tiêu chí:			
Điểm hướng dẫn:			

Cán bộ hướng dẫn

(Ký và ghi rõ họ tên)

ĐÁNH GIÁ ĐỒ ÁN TỐT NGHIỆP

(DÀNH CHO CÁN BỘ HƯỚNG DẪN)

Tên đề tài: **Xây dựng robot tự hành di động cùng với ROS sử dụng LiDAR 2D**

Họ tên sinh viên: **Bùi Việt Hoàng**

MSSV: **20203710**

Cán bộ hướng dẫn: **TS. Phạm Doãn Tĩnh**

STT	Tiêu chí (Điểm tối đa)	Hướng dẫn đánh giá tiêu chí	Điểm tiêu chí
1	Thái độ làm việc (2,5 điểm)	Nghiêm túc, tích cực và chủ động trong quá trình làm ĐATN	
		Hoàn thành đầy đủ và đúng tiến độ các nội dung được GVHD giao	
2	Kỹ năng viết quyển ĐATN (2 điểm)	Trình bày đúng mẫu quy định, bố cục các chương logic và hợp lý: Bảng biểu, hình ảnh rõ ràng, có tiêu đề, được đánh số thứ tự và được giải thích hay đề cập đến trong đồ án, có căn lề, dấu cách sau dấu chấm, dấu phẩy, có mở đầu chương và kết luận chương, có liệt kê tài liệu tham khảo và có trích dẫn, v.v.	
		Kỹ năng diễn đạt, phân tích, giải thích, lập luận: Cấu trúc câu rõ ràng, văn phong khoa học, lập luận logic và có cơ sở, thuật ngữ chuyên ngành phù hợp, v.v.	
3	Nội dung và kết quả đạt được (5 điểm)	Nêu rõ tính cấp thiết, ý nghĩa khoa học và thực tiễn của đề tài, các vấn đề và các giả thuyết, phạm vi ứng dụng của đề tài. Thực hiện đầy đủ quy trình nghiên cứu: Đặt vấn đề, mục tiêu đề ra, phương pháp nghiên cứu/ giải quyết vấn đề, kết quả đạt được, đánh giá và kết luận.	
		Nội dung và kết quả được trình bày một cách logic và hợp lý, được phân tích và đánh giá thỏa đáng. Biện luận phân tích kết quả mô phỏng/ phần mềm/ thực nghiệm, so sánh kết quả đạt được với kết quả trước đó có liên quan.	
		Chỉ rõ phù hợp giữa kết quả đạt được và mục tiêu ban đầu đề ra đồng thời cung cấp lập luận đề đề xuất hướng giải quyết có thể thực hiện trong tương lai. Hàm lượng khoa học/ độ phức tạp cao, có tính mới/tính sáng tạo trong nội dung và kết quả đồ án.	
4	Điểm thành tích (1 điểm)	Có bài báo KH được đăng hoặc chấp nhận đăng/ đạt giải SV NCKH giải 3 cấp Trường trở lên/ Các giải thưởng khoa học trong nước, quốc tế từ giải 3 trở lên/ Có đăng ký bằng phát minh sáng chế. (1 điểm)	
		Được báo cáo tại hội đồng cấp Trường trong hội nghị SV NCKH nhưng không đạt giải từ giải 3 trở lên/ Đạt giải khuyến khích trong cuộc thi khoa học trong nước, quốc tế/ Kết quả đồ án là sản phẩm ứng dụng có tính hoàn thiện cao, yêu cầu khối lượng thực hiện lớn. (0,5 điểm)	
Điểm tổng các tiêu chí:			
Điểm hướng dẫn:			

Cán bộ hướng dẫn

(Ký và ghi rõ họ tên)

Điểm từng tiêu chí cho lẻ đến 0,5. Nếu Điểm tổng các tiêu chí > 10 thì Điểm hướng dẫn làm tròn thành 10

ĐÁNH GIÁ ĐỒ ÁN TỐT NGHIỆP

(DÀNH CHO CÁN BỘ HƯỚNG DẪN)

Tên đề tài: **Xây dựng robot tự hành di động cùng với ROS sử dụng LiDAR 2D**

Họ tên sinh viên: **Bùi Việt Hoàng**

MSSV: **20203710**

Cán bộ hướng dẫn: **TS. Phạm Doãn Tĩnh**

STT	Tiêu chí (Điểm tối đa)	Hướng dẫn đánh giá tiêu chí	Điểm tiêu chí
1	Thái độ làm việc (2,5 điểm)	Nghiêm túc, tích cực và chủ động trong quá trình làm ĐATN Hoàn thành đầy đủ và đúng tiến độ các nội dung được GVHD giao	
2	Kỹ năng viết quyển ĐATN (2 điểm)	Trình bày đúng mẫu quy định, bố cục các chương logic và hợp lý: Bảng biểu, hình ảnh rõ ràng, có tiêu đề, được đánh số thứ tự và được giải thích hay đề cập đến trong đồ án, có căn lề, dấu cách sau dấu chấm, dấu phẩy, có mở đầu chương và kết luận chương, có liệt kê tài liệu tham khảo và có trích dẫn, v.v. Kỹ năng diễn đạt, phân tích, giải thích, lập luận: Cấu trúc câu rõ ràng, văn phong khoa học, lập luận logic và có cơ sở, thuật ngữ chuyên ngành phù hợp, v.v.	
3	Nội dung và kết quả đạt được (5 điểm)	Nêu rõ tính cấp thiết, ý nghĩa khoa học và thực tiễn của đề tài, các vấn đề và các giả thuyết, phạm vi ứng dụng của đề tài. Thực hiện đầy đủ quy trình nghiên cứu: Đặt vấn đề, mục tiêu đề ra, phương pháp nghiên cứu/ giải quyết vấn đề, kết quả đạt được, đánh giá và kết luận. Nội dung và kết quả được trình bày một cách logic và hợp lý, được phân tích và đánh giá thỏa đáng. Biện luận phân tích kết quả mô phỏng/ phần mềm/ thực nghiệm, so sánh kết quả đạt được với kết quả trước đó có liên quan. Chỉ rõ phù hợp giữa kết quả đạt được và mục tiêu ban đầu đề ra đồng thời cung cấp lập luận để đề xuất hướng giải quyết có thể thực hiện trong tương lai. Hàm lượng khoa học/ độ phức tạp cao, có tính mới/tính sáng tạo trong nội dung và kết quả đồ án.	
4	Điểm thành tích (1 điểm)	Có bài báo KH được đăng hoặc chấp nhận đăng/ đạt giải SV NCKH giải 3 cấp Trường trở lên/ Các giải thưởng khoa học trong nước, quốc tế từ giải 3 trở lên/ Có đăng ký bằng phát minh sáng chế. (1 điểm) Được báo cáo tại hội đồng cấp Trường trong hội nghị SV NCKH nhưng không đạt giải từ giải 3 trở lên/ Đạt giải khuyến khích trong cuộc thi khoa học trong nước, quốc tế/ Kết quả đồ án là sản phẩm ứng dụng có tính hoàn thiện cao, yêu cầu khối lượng thực hiện lớn. (0,5 điểm)	
Điểm tổng các tiêu chí:			
Điểm hướng dẫn:			

Cán bộ hướng dẫn

(Ký và ghi rõ họ tên)

LỜI CAM ĐOAN

Tôi tên là Bùi Việt Hoàng, mã số sinh viên 20203710, sinh viên lớp Điện tử 02, khóa 65. Giảng viên hướng dẫn là TS. Phạm Doãn Tình. Tôi xin cam đoan toàn bộ nội dung được trình bày trong đồ án "Xây dựng robot tự hành di động cùng với ROS sử dụng LiDAR 2D" là kết quả quá trình tìm hiểu và nghiên cứu của tôi. Các dữ liệu được nêu trong đồ án là hoàn toàn trung thực, phản ánh đúng kết quả đo đạc thực tế. Mọi thông tin trích dẫn đều tuân thủ các quy định về sở hữu trí tuệ; các tài liệu tham khảo được liệt kê rõ ràng. Tôi xin chịu hoàn toàn trách nhiệm với những nội dung được viết trong đồ án này.

Hà Nội, ngày 22 tháng 01 năm 2025

Người cam đoan

Bùi Việt Hoàng

Tóm tắt đồ án

Đồ án tốt nghiệp này tập trung trình bày một cách xây dựng một robot di động tự hành với hệ điều hành ROS (Robot operating system). Hệ thống sử dụng LiDAR 2D cùng gói module điều hướng 2D của ROS. Chức năng chính của robot đó là lập bản đồ bằng SLAM và tự điều hướng tránh vật cản trong không gian 2D.

Về phần mềm, em sử dụng nền tảng của ROS để có thể dễ dàng tương tác với các thành phần trong hệ thống. Về phần cứng, em gặp nhiều khó khăn với giới hạn của thiết bị và trong việc thiết lập hệ thống. Sản phẩm robot cuối cùng của em có thể tự điều hướng và linh động tránh vật cản. Hệ thống có thể được chia thành hai thành phần, module điều khiển robot và thu thập thông tin từ cảm biến được cài đặt trên ESP32 devkit và module thiết lập bản đồ cũng như điều hướng được thực hiện trên máy tính điều khiển sử dụng ROS.

Với đồ án này, em kỳ vọng bước đầu xây dựng được nền tảng để phát triển hệ thống robot di động tự hành để có thể ứng dụng trong các trường hợp thực tế để nâng cao năng suất lao động và chất lượng sản phẩm. Về hướng mở rộng, do yếu điểm của LiDAR 2D là chỉ có thể quét trên một mặt phẳng nằm ngang, giải pháp tích hợp thêm các cảm biến khác như Camera là một hướng đi hiệu quả để có thể khắc phục các thiếu sót của hệ thống.

Sinh viên thực hiện
Ký và ghi rõ họ tên

Abstract

This project focuses on presenting the implementation of an autonomous mobile robot with robot operation system. The system utilizes a 2D LiDAR with ROS navigation stack. The main functions of the robot are creating a 2-D occupancy grid map by using SLAM and navigating in 2D dimension including obstacles avoidance.

For software, I use ROS as a platform to conveniently interface the components in the system. Regarding hardware, the limitation of the devices and the system setup are some of my struggles. My final robot has the ability to navigate and dynamically avoid obstacles. The system can be divided into two parts, one is implemented on ESP32 devkit with a software module to collect data from sensors and a base controller; the second part is the SLAM and navigation module implemented on the control machine with ROS.

With this project, I strive to create a base in order to create an autonomous robot system that can be utilized in the real world to improve performance as well as product quality. In the future, to overcome one of the 2D LiDAR's drawbacks, which is the ability to only scan a single horizontal line, sensor fusion (e.g. camera) is an efficient way to enhance the adaptability of the system.

MỤC LỤC

CHƯƠNG 1. GIỚI THIỆU CHUNG	1
1.1 Lý do chọn đề tài.....	1
1.2 Mục tiêu của đề tài	2
CHƯƠNG 2. CƠ SỞ LÝ THUYẾT.....	3
2.1 Hệ điều hành Robot.....	3
2.2 Một số khái niệm trong ROS	3
2.2.1 Transform	3
2.2.2 Frame.....	4
2.2.3 Kết nối frame với transform.....	4
2.2.4 Odom, map và world frame.....	6
2.3 SLAM (Simultaneous Localization and Mapping).....	7
2.4 Phương pháp cục bộ hóa GMCL (General Monte Carlo Localization).....	8
CHƯƠNG 3. MÔ TẢ HỆ THỐNG	11
3.1 Tổng quan hệ thống.....	11
3.1.1 Hệ điều hành robot chạy trên máy chủ.....	11
3.1.2 Cảm biến và vi điều khiển.....	12
3.1.3 Thiết lập bản đồ.....	15
3.1.4 Định hướng.....	15
3.2 Triển khai hệ thống	18
3.2.1 Công cụ mô phỏng và mô hình hóa	18
3.2.2 Điều hướng cho robot sử dụng ROS và 2D LiDAR	18
3.2.3 Hành vi phục hồi của robot	20
CHƯƠNG 4. THỬ NGHIỆM VÀ KẾT QUẢ	22
4.1 Thử nghiệm	22
4.1.1 Thiết lập môi trường thử nghiệm	22
4.1.2 SLAM.....	22
4.1.3 Điều hướng.....	23
4.1.4 Kết quả	24
CHƯƠNG 5. KẾT LUẬN.....	25
TÀI LIỆU THAM KHẢO	26

DANH MỤC HÌNH VẼ

Figure 1. Một ví dụ về các frame của robot	4
Figure 2. Tính toán transform giữa các frame.....	5
Figure 3. Hệ phân cấp các frame.....	5
Figure 4. So sánh odom, map và world frame	6
Figure 5. Hoạt động của nút gmcl	9
Figure 6. Sơ đồ thuật toán của gmcl	10
Figure 7. Robot và các thiết bị	11
Figure 8. ROS Navigation stack của hệ thống	12
Figure 9. Sơ đồ nối dây các thiết bị trên robot.....	13
Figure 10. Cảm Biến gia tốc GY-521 6DOF IMU MPU6050.....	13
Figure 11. Laser Distance Sensor LDS-007.....	14
Figure 12. Sơ đồ quan hệ giữa các nút trong phần mềm.....	19
Figure 13. Sơ đồ góc nhìn từ trên cao cho navigation stack	20
Figure 14. Quá trình phục hồi của robot	21
Figure 15. Thiết lập cho thử nghiệm.....	22
Figure 16. Bản đồ được thiết lập nhờ SLAM thể hiện trên rviz	23
Figure 17. Điều hướng cho robot minh họa trên rviz.	24

DANH MỤC BẢNG BIỂU

Table 1. Các phương pháp và bộ lọc sử dụng	10
Table 2. Bảng cấu trúc tin nhắn Imu	14
Table 3. Bảng cấu trúc tin nhắn của LaserScan	15

CHƯƠNG 1. GIỚI THIỆU CHUNG

1.1 Lý do chọn đề tài

Robot tự hành đang ngày một trở nên phổ biến trong khắp các ngành công nghiệp do khả năng thực hiện các công việc được giao mà không cần nhiều sự can thiệp của con người. Robot có thể hoạt động trong các điều kiện và môi trường khắc nghiệt hoặc không thể tiếp cận được đối với con người. Robot tự hành thường được trang bị các cảm biến, camera và các thuật toán điều hướng để có khả năng đưa ra quyết định trong thời gian thực. Các chức năng đó cho phép chúng có khả năng thực hiện nhiều loại công việc, như vận chuyển đồ trong nhà kho, các dịch vụ giao hàng hay việc giám sát trong các môi trường phức tạp. Robot tự hành đặc biệt hữu dụng trong các ngành công nghiệp như logistics khi mà các công việc trong khâu vận hành được tối ưu về hiệu suất và giảm chi phí nhân công.

Việc xây dựng robot tự hành di động là một công việc đầy thử thách, yêu cầu kiến thức chuyên sâu và kỹ năng trong nhiều lĩnh vực khác nhau. Dưới đây là một số khó khăn gặp phải khi phát triển robot tự hành:

- **Khó khăn về cảm biến:** Chất lượng các cảm biến như LiDAR, camera, hay cảm biến siêu âm có thể có độ chính xác và độ phân giải hạn chế, ảnh hưởng đến khả năng nhận diện và đo lường môi trường xung quanh.
- **Tích hợp cảm biến:** Việc kết hợp dữ liệu từ nhiều cảm biến khác nhau (sensor fusion) để tạo ra một hình ảnh môi trường chính xác là một thách thức. Cảm biến có thể gặp phải hiện tượng nhiễu hoặc các yếu tố môi trường tác động (ví dụ: ánh sáng, thời tiết,...).
- **Điều khiển và ổn định:** Việc lập trình hệ thống điều khiển để robot có thể di chuyển mượt mà và chính xác, đồng thời tránh được các vật cản, là một thách thức lớn, đặc biệt khi di chuyển trên địa hình không đồng đều hoặc không gian chật hẹp.
- **Khử lỗi odometry:** Robot thường gặp phải sự sai lệch về vị trí do lỗi trong quá trình ước lượng vị trí (odometry). Điều này có thể làm robot di chuyển sai lệch, đặc biệt khi di chuyển dài mà không có sự hỗ trợ từ các hệ thống định vị ngoài trời như GPS.
- **Lập kế hoạch đường đi và điều hướng:** Việc lập kế hoạch cho robot tìm đường đi từ điểm A đến điểm B trong môi trường phức tạp, có chứa vật cản và thay đổi liên tục, đòi hỏi phải sử dụng các thuật toán phức tạp như A*, RRT (Rapidly-exploring Random Trees).
- **Điều hướng trong môi trường không xác định:** Robot phải có khả năng phát hiện và xử lý các thay đổi trong môi trường như sự xuất hiện của vật cản mới, thay đổi địa hình, hoặc các yếu tố tác động khác mà không bị mất phương hướng.
- **Tính toán và tối ưu hóa hiệu suất:** Việc đảm bảo robot sử dụng tài nguyên (như năng lượng, bộ vi xử lý) một cách tối ưu là một thách thức lớn, nhất

là khi robot phải hoạt động liên tục và kéo dài thời gian sử dụng mà không cần sạc lại.

- An toàn khi vận hành: Robot phải có các hệ thống bảo vệ để tránh va chạm với người, vật thể, hoặc các môi trường nguy hiểm. Điều này đặc biệt quan trọng khi robot hoạt động trong môi trường chung với con người.
- Chi phí phát triển: Việc phát triển một robot tự hành di động đòi hỏi rất nhiều chi phí, từ phần cứng (cảm biến, động cơ, khung gầm) đến phần mềm (thuật toán, hệ điều hành). Chi phí này có thể là một yếu tố hạn chế trong việc xây dựng và triển khai.
- Kiểm tra và thử nghiệm thực tế: Việc kiểm tra và tinh chỉnh robot trong môi trường thực tế là rất quan trọng và khó khăn. Các thử nghiệm có thể gặp phải sự cố bất ngờ mà bạn không thể lường trước, và việc sửa lỗi có thể mất nhiều thời gian.

Từ các lý do trên, đề án tốt nghiệp của em cố gắng đưa ra một giải pháp để bước đầu phát triển một hệ thống ứng dụng trên robot di động tự hành mà có thể tự động điều hướng và có thể tránh khỏi các vật cản để đảm bảo an toàn cho tài sản và con người.

1.2 Mục tiêu của đề tài

Mục tiêu chung của dự án là phát triển một robot tự hành di động với hệ điều hành robot (ROS). Hệ thống sử dụng LiDAR 2D cùng với ROS 2D Navigation stack với chức năng chính là thiết lập bản đồ sử dụng phương pháp SLAM (Simultaneous Localization and Mapping) dựa trên cảm biến laser và chức năng định hướng (navigation). Hệ thống hướng đến sự tiết kiệm năng lượng tiêu thụ và một vi xử lý nằm trên robot có giá thành thấp. Ưu tiên hàng đầu của hệ thống là tính an toàn đối với con người và tài sản.

Với những mục tiêu đó, khi chọn đề tài này em mong muốn học được nhiều kiến thức và kinh nghiệm về hệ thống robot và điều khiển, về động học và động lực học, hệ thống điều khiển như PID (Proportional-Integral-Derivative). Ngoài ra, em cũng học được cách sử dụng các cảm biến như LiDAR, IMU (Inertial Measurement Units) để có thể thu thập dữ liệu từ môi trường xung quanh cũng như cách kết hợp dữ liệu từ nhiều cảm biến để có thể nâng cao hiệu năng và độ chính xác. Dự án này cũng giúp em hiểu sâu hơn về hệ thống nhúng, lập trình nhúng cho vi điều khiển và tương tác với nhiều loại linh kiện điện tử khác nhau.

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT

2.1 Hệ điều hành Robot

ROS là một framework mã mở cung cấp cho người dùng các công cụ, thư viện và mô hình để phát triển phần mềm dành cho robot [1]. ROS là một nền tảng được sử dụng rộng rãi trong việc xây dựng robot [2]. Nó không phải một hệ điều hành theo đúng nghĩa (như Windows hay Linux), mà là một phần mềm cung cấp các nền tảng để phần mềm robot có thể dễ dàng tích hợp với các thành phần khác trong hệ thống như cảm biến, bộ điều khiển hay thuật toán. Hơn thế nữa, ROS cung cấp các API để xây dựng các gói phần mềm hoặc kết nối với các hệ thống bên ngoài hoặc các thành phần khác [3].

Một số đặc điểm nổi bật của ROS:

- **Tính module:** ROS khuyến khích việc chia quá trình phát triển ra thành nhiều module có thể tái sử dụng bằng cách chia các hành vi của robot thành các node cá biệt. Điều này giúp việc viết, kiểm tra và bảo trì phần mềm của các hệ thống phức tạp dễ dàng hơn.
- **Giao tiếp:** ROS sử dụng hệ thống giao tiếp xuất bản-đăng ký (publish-subscribe) cho quá trình truyền tin giữa các process. Các Node có thể xuất bản thông tin (như thông tin từ cảm biến) hoặc đăng ký để nhận loại thông tin đó, cho phép dòng chảy dữ liệu trong hệ thống trở nên mượt mà hơn.
- **Tính trừu tượng:** ROS giúp trừu tượng hóa các chi tiết cấp thấp như các giao diện phần cứng và cho phép người dùng tương tác với robot ở một cấp cao hơn.
- **Giả lập:** ROS có các công cụ như Gazebo và RVIZ cung cấp môi trường giả lập 3D cho phép thử nghiệm robot trên máy tính cũng như cho phép nhìn trực quan quá trình hoạt động của robot trên màn hình.
- **Khả năng tương thích đa nền tảng:** ROS có thể chạy trên cả Linux và Windows. Nó được thiết kế để có thể chạy trên nhiều thiết lập phần cứng khác nhau mà không cần thay đổi nhiều về phần mềm.

2.2 Một số khái niệm trong ROS

2.2.1 Transform

Robot là một hệ thống phức hợp được điều hướng trong môi trường thực phức tạp. Để có thể mô tả cách mà chúng tương tác với các thành phần khác nhau trong môi trường, các phép toán học được sử dụng gọi là transformations hay gọi tắt là transforms. Nói đơn giản, transforms thể hiện vị trí và hướng của một vật thể trong quan hệ với một vật thể khác.

Trong ROS, tin nhắn transform có tên *geometry_msgs/Transform.msg* bao gồm một vector tọa độ trong không gian và một vector thể hiện hướng trong hệ quaternion.

2.2.2 Frame

Trong robot học, một frame ám chỉ tới hệ tọa độ miêu tả vị trí của một vật thể và hướng của nó, thường là trong các trục x,y và z. ROS yêu cầu mỗi frame cần có một mã nhận diện tương ứng (frame_id). Mỗi hệ thống robot thường bao gồm nhiều frame cho mỗi thành phần khác nhau (khung robot, cảm biến, tay quay,...), các vật thể trong môi trường cũng được miêu tả bằng frame.

Trong mọi trường hợp, hệ thống đều nằm trong một frame cơ bản, thường được gọi là *world* hay *map* – đây là frame không thay đổi. Mọi frame khác trong bối cảnh, kể cả frame của robot mà ROS thường gọi là *base_link* – đều là con của frame chính và được định vị tương đối so với frame chính.

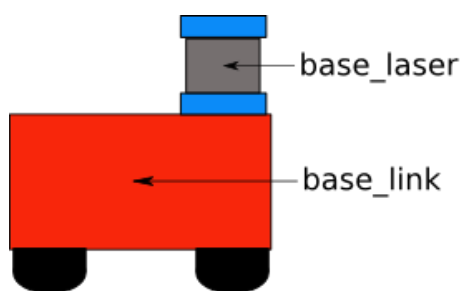


Figure 1. Một ví dụ về các frame của robot

Base_link của robot thường bắt đầu ở thân của nó và phát triển ra các frame con cho các cảm biến của robot, ví dụ như *base_laser*.

2.2.3 Kết nối frame với transform

Transform quyết định sự biến dịch chuyển và phép quay cần thiết để có thể dịch chuyển từ một frame nguồn tới frame đích. Một bộ transform hoàn chỉnh của một bối cảnh, từ frame chính cho tới các frame con cấu thành nên một **transform tree**.

Với transform tree, ta có thể xác định được vị trí cũng như hướng quay của bất kỳ frame nào trong bối cảnh. Transform có thể có dạng tĩnh hoặc động (static transform and mobile transform). Dạng tĩnh ví dụ như giữa *base_link* và *base_laser* sẽ không thay đổi, trong khi dạng động ví dụ như *base_link* so với một frame di chuyển tương đối với nó có thể sẽ thay đổi trong các thời điểm khác nhau.

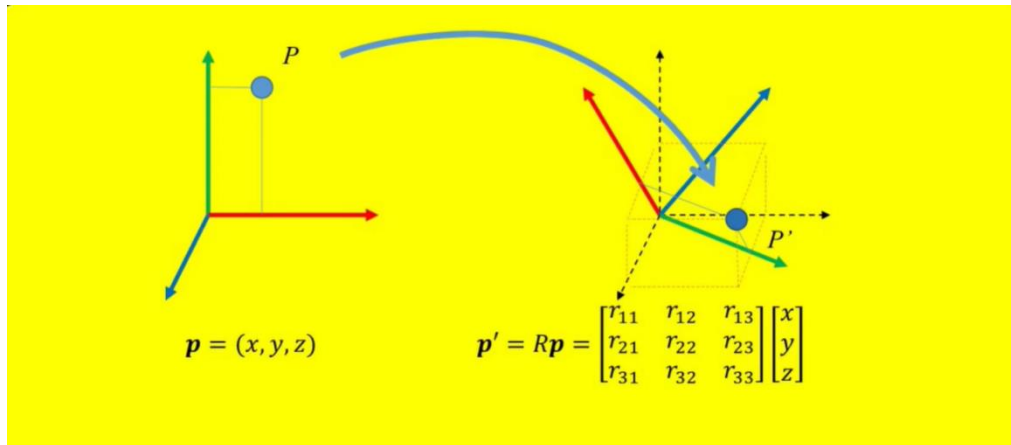


Figure 2. Tính toán transform giữa các frame

ROS cung cấp thư viện “tf” như một cách để có thể dễ dàng tính toán transform, hình trên thể hiện một phần cách mà thư viện này thực hiện việc đó. Ví dụ để tính toán transform từ frame P đến frame P’, ta cần hai tham số:

- R - ma trận transform 2D giữa frame cha và frame P.
- p - Vector vị trí của frame P trong quan hệ tương đối với frame P’.

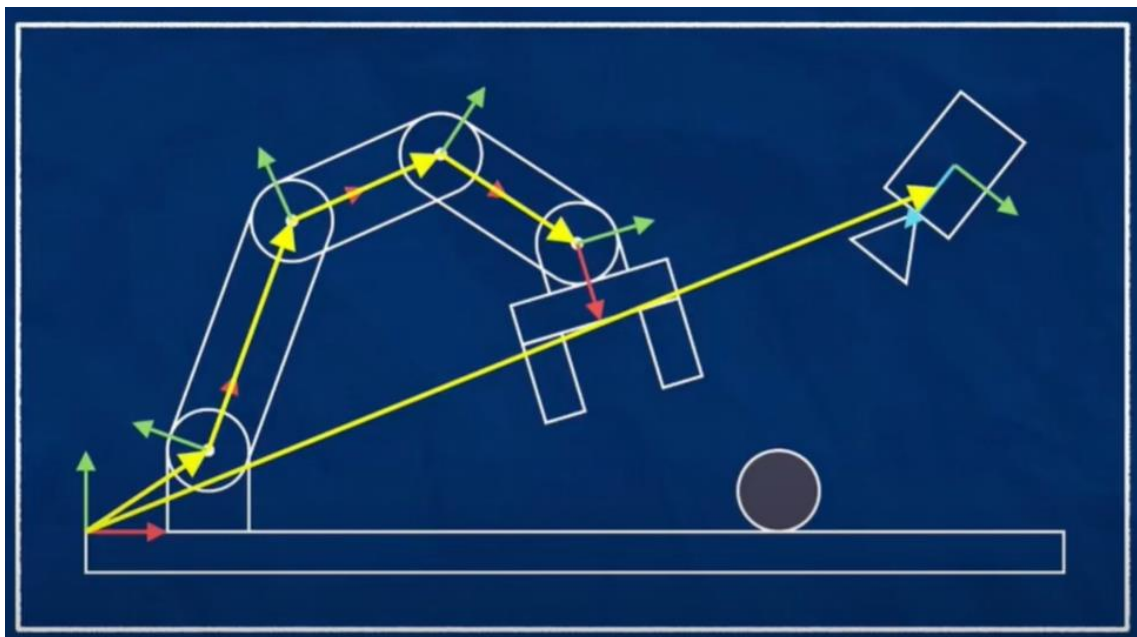


Figure 3. Hệ phân cấp các frame. Ở góc dưới bên trái là world frame và các frame con được kết nối với frame này bằng các transform miêu tả bởi mũi tên màu vàng.

Để đơn giản hóa các hệ thống phức tạp trong thực tế, ta tạo ra một hệ cấp bậc gồm các hệ thống con mà trong đó lưu trữ transform của frame con tới frame cha, từ đó làm cho việc xác định vị trí của các thành phần trong bản đồ dễ dàng hơn trong trường hợp trạng thái của frame cha có sự thay đổi.

Trong ROS, các tin nhắn transform được truyền tin trong hai chủ đề, /tf_static cho transform tĩnh và /tf cho transform động [4]. Sự tách biệt này giúp cải thiện băng thông và giảm số lượng tin nhắn transform phải trao đổi.

2.2.4 Odom, map và world frame

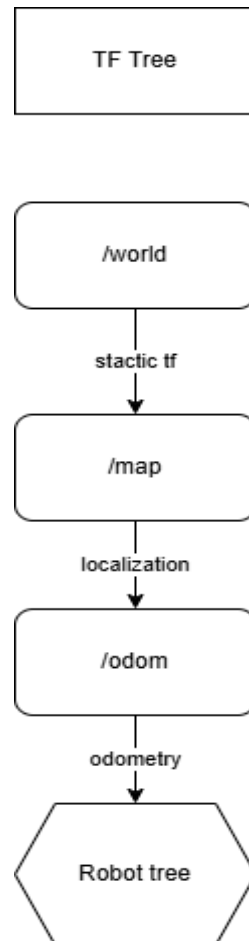


Figure 4. So sánh odom, map và world frame

Hình trên thể hiện một ví dụ của cây transform của một hệ thống robot. Frame của khung đế robot hay được gọi là base_link. Như đã đề cập ở trên, cấp độ của cây transform lần lượt là /world, /map, /odom và /base_link. Đó là một số những frame cơ bản hay được sử dụng trong cây transform của ROS. World frame được gọi là frame cha nằm ở gốc của cây. Map frame được tạo nên bởi nút “map_server” định nghĩa một hệ tọa độ cho môi trường mà robot đang hoạt động ở trong đó, là một frame tĩnh tương đối với world frame. Odom frame là khung mà ở đó robot bắt đầu quá trình định hướng.

Quá trình tính toán transform của /map đến /odom được gọi là cục bộ hóa (localization). Quá trình tính toán transform từ /odom đến /base_link được gọi là quá trình ước lượng vị trí (odometry). Có nhiều thuật toán có thể được áp dụng cho quá trình cục bộ hóa và ước lượng vị trí [5].

2.3 SLAM (Simultaneous Localization and Mapping)

SLAM (Simultaneous Localization and Mapping) là một kỹ thuật căn bản được sử dụng nhiều trong ngành robot cho phép xây dựng một bản đồ của một môi trường chưa biết trước trong khi đồng thời theo dõi vị trí của robot trên bản đồ đó. Kỹ thuật này rất cần thiết để robot có thể hoạt động trong nhiều môi trường mới khi mà thông tin về bản đồ không có sẵn.

Những khái niệm căn bản trong SLAM:

- **Localization:** Robot cần ước tính vị trí của nó trong môi trường khi nó di chuyển, sử dụng các loại cảm biến (như là LiDAR, camera hay cảm biến độ sâu) để thu thập thông tin về không gian xung quanh.
- **Mapping:** Trong khi robot di chuyển, nó xây dựng một bản đồ dựa trên thông tin thu thập được từ các cảm biến. Bản đồ có thể có nhiều dạng như là bản đồ 2D, 3D hoặc một tập hợp các đặc điểm của môi trường.
- **Simultaneous:** Thử thách quan trọng trong SLAM đó là robot phải làm cả hai việc – localization và mapping – cùng một lúc. Robot dùng thông tin từ các cảm biến để dần dần xây dựng bản đồ và liên tục cập nhật vị trí của nó trong môi trường trong khi phải đối diện với các sai khác trong thực tế (như là nhiễu của cảm biến, các sai lệch trong di chuyển hoặc các thay đổi của môi trường).

Cách SLAM hoạt động:

1. **Xuất phát từ vị trí không biết ban đầu:** robot được khởi động ở một vị trí mà nó không biết trước trong môi trường.
2. **Thu thập thông tin từ môi trường:** Khi robot di chuyển xung quanh, nó thu thập thông tin từ cảm biến (LiDAR, camera,...) và sử dụng thông tin đó để phát hiện các đặc điểm hoặc cột mốc trong môi trường.
3. **Mapping và Localization:**
 - **Ước tính vị trí:** Robot thực hiện ước tính vị trí và hướng của nó trên bản đồ dựa vào quá trình di chuyển trước đó của nó và thông tin từ cảm biến. Một số thuật toán hay được sử dụng ở đây là **particle filters** (AMCL) hoặc **Kalman filters**.
 - **Cập nhật bản đồ:** Robot cập nhật lại bản đồ bằng cách thêm các thông tin mới nhận từ cảm biến, liên tục hoàn thiện bản đồ và các cột mốc trong đó.

4. **Đóng vòng lặp:** Khi robot di chuyển xung quanh, nó sẽ gặp lại các khu vực đã đi qua. Nếu robot có thể nhận diện được khu vực đó, nó sẽ sửa lại các sai lệch trước đó và hoàn thiện bản đồ. Quá trình này còn được biết là đóng vòng lặp (loop closure).

2.4 Phương pháp cục bộ hóa GMCL (General Monte Carlo Localization)

Gmcl (General Monte Carlo Localization) là một kỹ thuật nội địa hóa (localization) dựa trên xác suất dành cho robot di động trong bản đồ 2D. Phương pháp này tích hợp phương pháp **adaptive monte carlo localization** - amcl – và thêm 3 thuật toán bộ lọc hạt (particle filter) khác nhau: Optimal, Intelligent, Self-adaptive để cải thiện hiệu suất khi hoạt động trong thời gian thực [6].

Bộ lọc hạt hay còn được gọi là phương pháp Monte Carlo tuần tự là một kỹ thuật áp dụng dựa trên bộ lọc Bayes, nhằm ước tính một cách đệ quy trạng thái của hệ thống, đặc biệt đối với các hệ thống có nhiễu không tuyến tính và không tuân theo phân phối Gauss. Giả sử ta muốn ước tính trạng thái x_t của một hệ thống tại thời gian t , với mô hình quan sát (ví dụ như LiDAR) là $z_{1:t} = \{z_1, z_2, \dots, z_t\}$, mô hình điều khiển là $u_{1:t} = \{u_1, u_2, \dots, u_t\}$. Hệ thống được giả định tuân theo mô hình chuyển đổi trạng thái (state transition model), tại thời điểm t trạng thái của hệ thống là:

$$x_t = f(x_{t-1}, u_t) + \eta_t$$

- $f()$ là hàm không tuyến tính thể hiện sự chuyển đổi trạng thái.
- η_t là nhiễu trong quá trình chuyển đổi trạng thái.

Đối với bộ lọc Bayes, mục tiêu chính là tính toán được xác suất hậu nghiệm của một trạng thái tại một thời điểm t , biết thông tin quan sát được cho tới thời điểm đó – $p(x_t | z_{1:t})$. Bộ lọc Bayes tính xác suất hậu nghiệm dựa trên sự ước tính trạng thái trước đó:

$$p(x_t | z_{1:t}) = \frac{p(z_t | x_t)p(x_t | z_{1:t-1})}{p(z_t | z_{1:t-1})}$$

- $p(x_t | z_{1:t-1})$ là dự đoán dựa vào xác suất hậu nghiệm tại thời điểm trước đó.
- $p(z_t | x_t)$ là xác suất của mô hình quan sát được.
- $p(z_t | z_{1:t-1})$ là hằng số thông thường hóa (normalizing constant).

Việc tính toán xác suất hậu nghiệm trên thường gặp nhiều khó khăn do tính phi tuyến của nhiễu không tuân theo phân phối Gauss, bộ lọc hạt tính toán phân phối này bằng cách xấp xỉ nó theo một bộ các hạt (particle) $\{x_t^{[i]}\}$ để thể hiện cho phân phối đó. Có ba bước chính được áp dụng trong thuật toán này (được lặp lại):

1. Lấy mẫu các hạt từ mô hình chuyển động:

$$x_t^{[i]} \sim p(x_t | x_{t-1}, u_t)$$
2. Tính toán trọng số của mỗi hạt dựa trên mô hình quan sát:

$$w_t^{[i]} \sim p(z_t | x_t)$$

3. Các hạt được lấy mẫu lại dựa theo trọng số, sao cho các hạt với trọng số lớn hơn có khả năng được tồn tại lâu hơn, và các hạt với trọng số nhỏ hơn bị loại bỏ.

Trong ROS, GMCL node được phát triển dựa trên thuật toán amcl của tác giả Brian Gerkey. Giống với amcl, gmcl nhận đầu vào là: map dựa vào laser, laser scan, và transform messages và cho đầu ra là ước tính vị trí hiện tại (pose estimates). Hình dưới miêu tả quá trình localization sử dụng gmcl. Trong quá trình thực thi, gmcl ước tính sự biến đổi của base frame so với map frame nhưng chỉ xuất bản sự biến đổi (transform) giữa map frame và odometry frame. Về cơ bản có thể nói, sự biến đổi này thể hiện sự xê dịch (drift) trong quá trình dead reckoning [7].

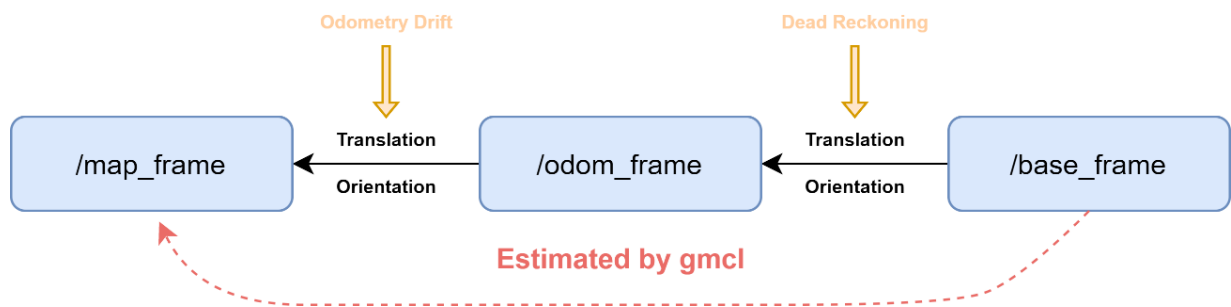


Figure 5. Hoạt động của nút gmcl

Điểm nổi bật của gmcl so với amcl nằm ở ba thuật toán mới: **Optimal particle filter**, **Intelligent particle filter** và **Self-adaptive particle filter** để cải thiện hiệu suất khi hệ thống phải chạy trong thời gian thực. Cách tích hợp các thuật toán này với amcl, Pseudo code của gmcl, phân tích so sánh về hiệu năng và độ phức tạp của hai phương thức amcl và gmcl được miêu tả rõ ràng trong bài báo [8].

Gmcl cho phép cấu hình việc sử dụng hoặc vô hiệu hóa đối với ba bộ lọc mới:

- **Optimal particle filter:** gmcl sẽ tính toán trọng số của các hạt phụ trợ được thêm vào của bộ lọc tối ưu.
- **Intelligent particle filter:** gmcl sẽ áp dụng mô hình chuyển biến chéo (crossover and mutation) dành cho vị trí của các hạt có trọng số nhỏ.
- **Self-adaptive particle:** gmcl sẽ phân bố các hạt một cách ngẫu nhiên trong vùng Similar Energy Region (SER), được chứng minh trong bài báo [9] có hiệu suất tốt hơn trong quá trình cục bộ hóa.

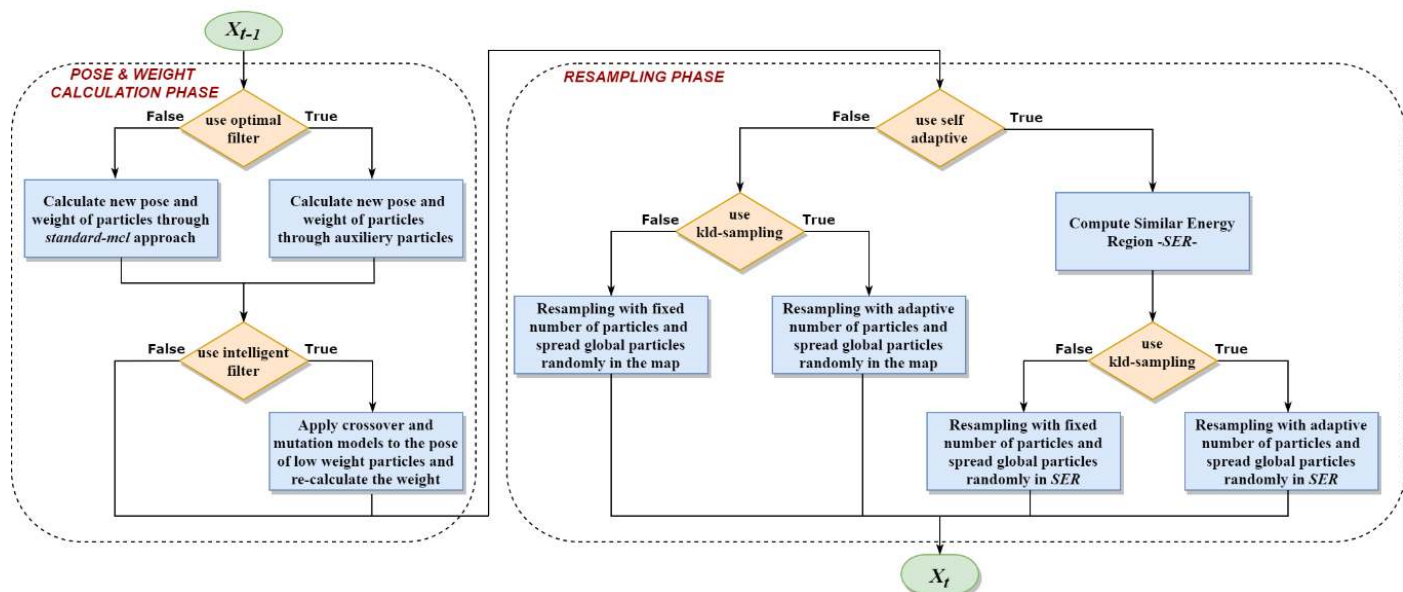


Figure 6. Sơ đồ thuật toán của gmcl [12]

Bảng sau thể hiện các bộ lọc được thực hiện của từng phương pháp khác nhau. Có thể thấy rằng gmcl có thể thực hiện được với các loại bộ lọc giống như amcl:

Tên phương pháp	Optimal articlel filter	Intelligent particle filter	Self-adaptive particle filter	KLD-sampling filter
Augmented-mcl	0	0	0	0
amcl	0	0	0	1
gmcl	x	x	x	x

Table 1. Các phương pháp và bộ lọc sử dụng

CHƯƠNG 3. MÔ TẢ HỆ THỐNG

3.1 Tổng quan hệ thống

3.1.1 Hệ điều hành robot chạy trên máy chủ

Về phần phần mềm, em sử dụng ROS là nền tảng chính để triển khai cả hệ thống. Hệ thống ROS được chạy trên máy chủ giao tiếp với robot qua kết nối không dây (websocket).

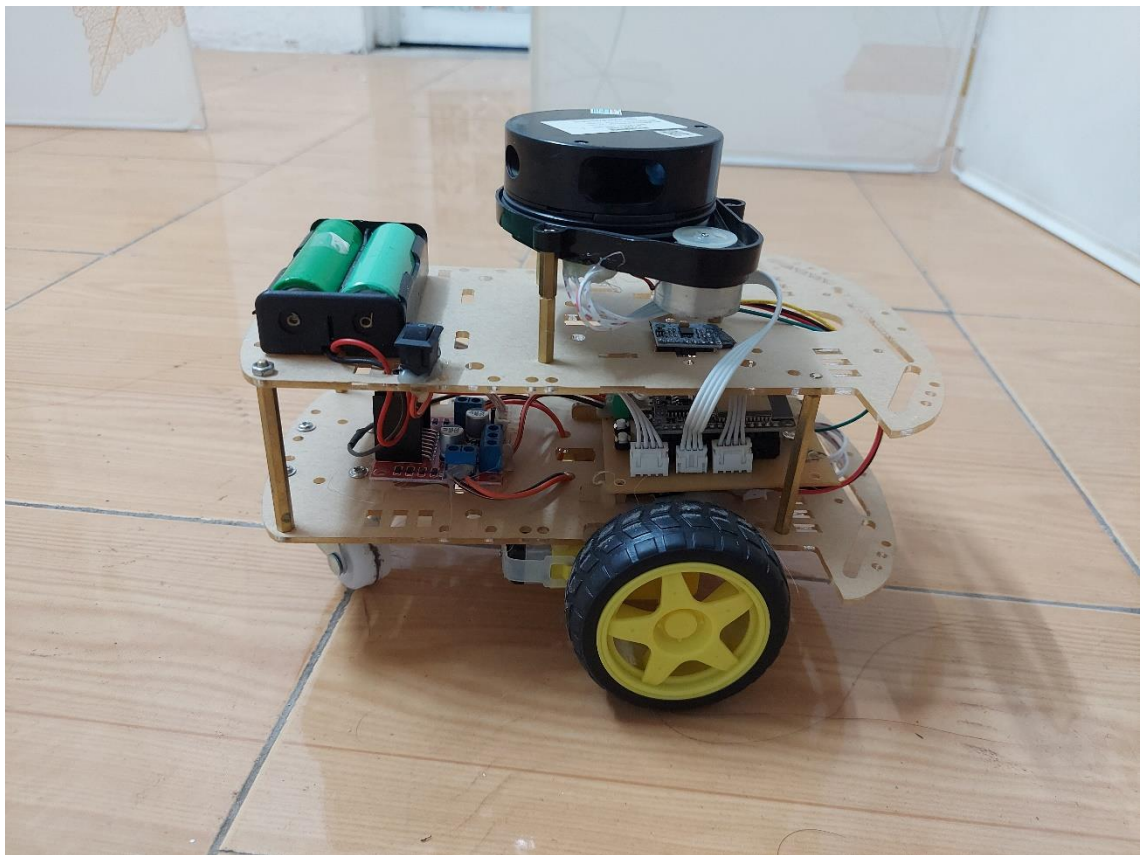


Figure 7. Robot và các thiết bị

Các gói phần mềm được sử dụng trong hệ thống là:

- “tf” Đây là gói phần mềm giúp theo dõi các frame trong các thời điểm.
- “slam_gmapping” Đây là gói phần mềm thực hiện quá trình SLAM
- “map_server” Đây là gói phần mềm cung cấp dữ liệu bản đồ của ROS.
- “gmcl” Đây là gói phần mềm cho quá trình cục bộ hóa
- “move_base” thực hiện quá trình điều khiển robot đến điểm đích.

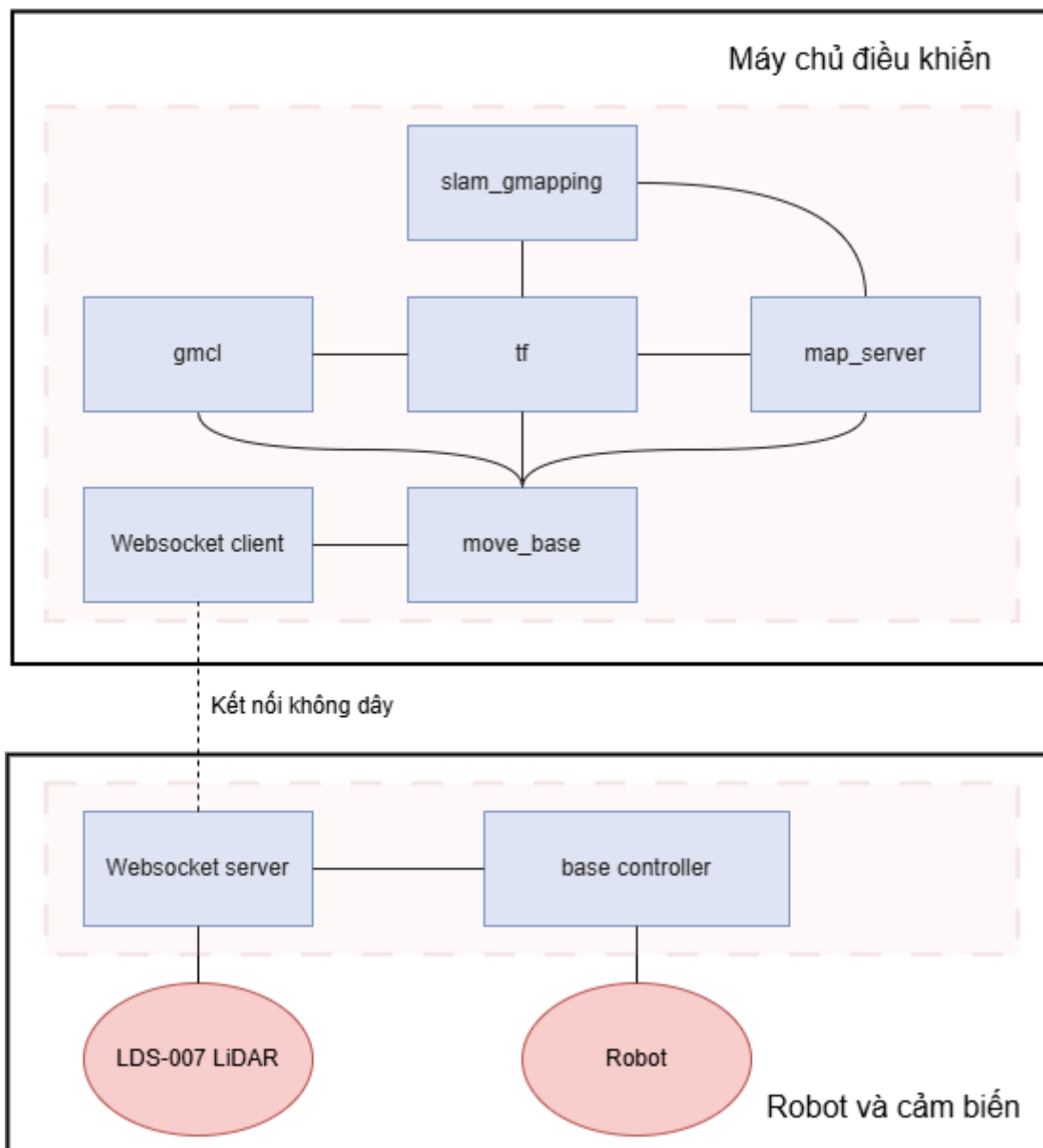


Figure 8. ROS Navigation stack của hệ thống

3.1.2 Cảm biến và vi điều khiển

Cho phần thân của robot tự hành, LiDAR 2D được đặt trên robot và cảm biến gia tốc GY-521. Cảm biến LiDAR được hiệu chỉnh và đặt cách mặt đất khoảng 20 cm. Một vi điều khiển được đặt trên robot với nhiệm vụ chính để lấy dữ liệu từ các cảm biến, gửi cho máy chủ qua giao tiếp không dây và nhận lệnh từ máy chủ để điều khiển robot. Các thiết bị được dùng được liệt kê dưới đây:

- LiDAR 2D model LDS-007.

- Bộ đo lường quán tính GY-521 6DOF IMU MPU6050.
- Vi điều khiển ESP32-DEVKIT-V1.

Vi điều khiển được dùng là loại tiêu thụ năng lượng ít và giá thành không cao. Nó được kết nối với nguồn cung cấp 30 WH và giao tiếp với máy chủ qua mạng không dây. Tổng quan về cơ cấu sắp đặt của các thiết bị cùng với robot được thể hiện qua hình dưới đây và sơ đồ kết nối ở hình 9.

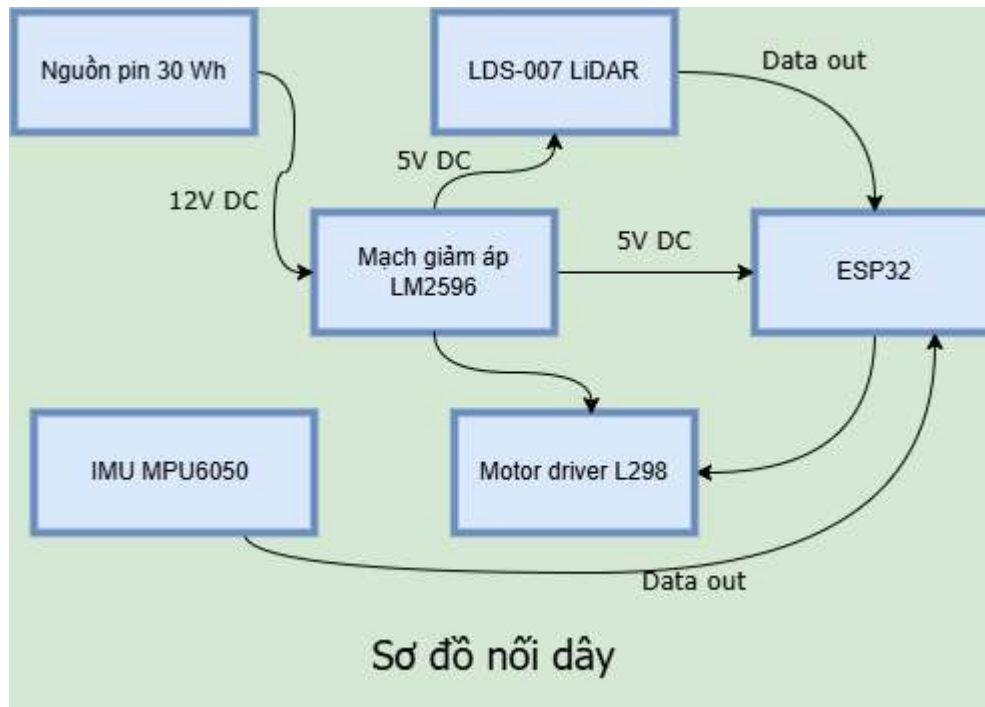


Figure 9. Sơ đồ nối dây các thiết bị trên robot

IMU (Inertial unit measurement) cung cấp các thông tin đo lường về gia tốc, vận tốc và góc quay. Trong hệ thống này, chỉ có thông tin về góc quay được quan tâm, cụ thể là góc quay xung quanh trục z (trục thẳng đứng so với mặt đất) hay còn



Figure 10. Cảm Biến gia tốc GY-521 6DOF IMU MPU6050

được gọi là yaw. IMU cần được gắn chắc ở trên đầu của robot vì nếu cảm biến bị rung động trong quá trình di chuyển sẽ ảnh hưởng đến quá trình đo lường.

Cấu trúc tin nhắn được dùng trong ROS để chứa dữ liệu của một IMU có tên là sensor_msgs/Imu Message được thể hiện qua bảng sau:

Kiểu dữ liệu	Tên biến	Ghi chú
Header	header	
geometry_msgs/Quaternion	orientation	Thể hiện hướng trong không gian ở dạng Quaternion
float64[9]	orientation_covariance	
geometry_msgs/Vector3	angular_velocity	
float64[9]	angular_velocity_covariance	
geometry_msgs/Vector3	linear_acceleration	
float64[9]	linear_acceleration_covariance	

Table 2. Bảng cấu trúc tin nhắn Imu

LiDAR cung cấp thông tin về độ sâu hay khoảng cách giữa robot và các vật thể xung quanh. LiDAR được đặt trên đỉnh của robot để có được tầm nhìn toàn diện nhất, không bị ngăn cản. Trong hệ thống, mẫu LDS-007 được sử dụng với khoảng cách đo được tối đa là 5.5 mét.



Figure 11. Laser Distance Sensor LDS-007

Cấu trúc tin nhắn dữ liệu của LiDAR scan (sensor_msgs/LaserScan.msg) được định nghĩa trong ROS được thể hiện qua bảng dưới đây:

Kiểu dữ liệu	Tên biến	Ghi chú
Header	header	Biến timestamp trong header là thời điểm tia quét đầu tiên được thực hiện.
float32	angle_min	Góc bắt đầu của tia đo [rad]
float32	angle_max	Góc kết thúc của tia đo [rad]
float32	angle_increment	Khoảng cách giữa các góc của tia đo [rad]
float32	time_increment	Khoảng thời gian giữa các tia đo [s]
float32	scan_time	Khoảng thời gian giữa các lần quét [s]
float32	range_min	Giá trị khoảng cách tối thiểu [m]
float32	range_max	Giá trị khoảng cách tối đa [m]
float32[]	ranges	Dữ liệu khoảng cách [m]
float32[]	intensities	Dữ liệu cường độ

Table 3. Bản cấu trúc tin nhắn của LaserScan

Sau khi các thiết bị được kết nối với ROS, bước chuẩn bị tiếp theo là một bản đồ tĩnh. Trước khi quá trình thiết lập bản đồ có thể được thực hiện trên ROS, ta phải thiết lập các tọa độ transform cho các cảm biến với robot. Trong hệ thống này, em sử dụng gói “tf” cùng với loại static transform (chi tiết tại <http://wiki.ros.org/tf>). Bằng cách này, ta có thể có được tọa độ transform của LiDAR so với đế nền của robot. ROS trợ giúp trong việc thiết lập sự biến đổi tọa độ và động học (kinematic and coordinate transformation).

3.1.3 Thiết lập bản đồ

Trong hệ thống này, em sử dụng bản đồ tĩnh để làm cột mốc cho việc định hướng. Một mục tiêu hay điểm đến mà ta gửi cho robot sẽ được thể hiện trên tọa độ của bản đồ tĩnh này. Trong ROS, hệ điều hành cung cấp nhiều gói phần mềm SLAM dựa trên dữ liệu quét được của LiDAR 2D với nhiều cách cài đặt khác nhau. Em chọn gói phần mềm “slam_gmapping”. Với gói này, bản đồ 2D được thiết lập từ LiDAR của robot. Để thu thập dữ liệu cho SLAM, em sử dụng gói phần mềm “teleop_twist_keyboard” để điều khiển robot thông qua mạng không dây.

3.1.4 Định hướng

Robot sẽ sử dụng hai loại hoạch định (planner) để di chuyển trên bản đồ, đó là *Global* và *Local*:

- Hoạch định global thiết lập một đường đi tới một mục tiêu trên bản đồ hoặc đến một địa điểm cách tương đối xa sao cho đường đi đặt ra tránh khỏi các vật cản sẵn có hoặc vật cản mới phát hiện với tần số thiết lập đường đi mới được chỉ định trong tham số planner_frequency. Trong hệ

thống này, thuật toán được dùng để định hướng global là thuật toán **Dijkstra**.

- Hoạch định local cung cấp một controller để có thể điều khiển robot theo quỹ đạo đến điểm đích. Khi di chuyển trên quỹ đạo đó, thuật toán tạo ra các hàm giá trị, được thể hiện trên một bản đồ lưới. Hàm giá trị này giúp controller tính toán được vận tốc tối ưu để gửi cho robot. Trong hệ thống này, thuật toán được sử dụng để định hướng local là **Dynamic Window Approach (DWA)**.

Thuật toán **Dynamic Window Approach** được thể hiện qua các bước sau:

1. Lấy mẫu rời rạc trong không gian xung quanh robot (dx,dy,dtheta)
2. Cho mỗi mẫu vận tốc rời rạc, thực hiện mô phỏng để robot đi theo vận tốc đó.
3. Đánh giá các quỹ đạo trong quá trình mô phỏng dựa trên các tiêu chí: khoảng cách tới vật cản, khoảng cách tới đích, sự trùng lặp so với đường đi global và tốc độ. Các quỹ đạo va chạm với vật cản bị loại bỏ.
4. Chọn quỹ đạo có giá trị tốt nhất và gửi vận tốc kèm theo tới controller
5. Lặp lại từ đầu

Hai loại hoạch định này sử dụng bản đồ giá trị (costmap) để lưu thông tin. Có 2 loại costmap tương đương là global costmap và local costmap. Costmap có các tham số để cấu hình cho hành vi của việc điều hướng, và chúng có các tham số chung với nhau, được cấu hình ở 3 file:

- costmap_common_params.yaml
- global_costmap_params.yaml
- local_costmap_params.yaml

Cấu hình tham số chung

Sau đây là các tham số trong file costmap_common_params.yaml:

```
obstacle_range: 2.5
raytrace_range: 3.0
footprint: [[-0.2,-0.2],[-0.2,0.2], [0.2, 0.2], [0.2,-0.2]]
inflation_radius: 0.5
cost_scaling_factor: 10.0
observation_sources: scan
scan: {sensor_frame: base_link, observation_persistence: 0.0, max_
obstacle_height: 0.4, min_obstacle_height: 0.0, data_type: LaserScan,
topic: /scan, marking: true, clearing: true}
```

File này dùng để cấu hình các tham số chung được sử dụng trong local_costmap và global_costmap.

- obstacle_range và raytrace_range dùng để chỉ khoảng cách lớn nhất mà cảm biến có thể đọc và cung cấp thông tin trong costmap, ví dụ trong trường hợp trên là 2.5 mét.
- Footprint dùng để chỉ kích thước hình học của robot cho navigation stack. Nó dùng để giữ khoảng cách hợp lý giữa vật cản và robot hoặc tính toán khả năng đi qua chỗ hẹp. inflation_radius là giá trị chỉ khoảng cách tối thiểu cho phép của robot với vật cản.
- cost_scaling_factor chỉ hành vi của robot khi di chuyển xung quanh vật cản, có thể điều chỉnh cho robot trở nên cẩn thận hoặc ngược lại.
- observation_sources dùng để chọn cảm biến được dùng cho navigation stack để lấy dữ liệu từ môi trường thực tế và tính toán đường đi, trong trường hợp của dự án này ta dùng LiDAR.

Dòng cuối cùng dùng để cấu hình frame của cảm biến và loại data sử dụng.

Cấu hình tham số global costmap

Sau đây là các tham số trong file global_costmap_params.yaml

```
global_costmap:  
global_frame: /map  
robot_base_frame: /base_footprint  
update_frequency: 1.0  
static_map: true
```

- global_frame và robot_base_frame định nghĩa sự biến đổi giữa bản đồ và vị trí của robot sử dụng cho global map.
- update_frequency cho phép cấu hình tần số cập nhật bản đồ.

Cấu hình tham số cho local costmap

Sau đây là các tham số trong file local_costmap_params.yaml:

```
local_costmap:  
global_frame: /map  
robot_base_frame: /base_footprint  
update_frequency: 3.0  
publish_frequency: 2.0  
static_map: false  
rolling_window: true  
width:  
height:  
resolution:  
transform_tolerance: 0.5
```

planner_frequency: 1.0

planner_patiente: 5.0

- global_frame, robot_base_frame, update_frequency tương tự với các tham số trong phần global costmap. Tham số rolling_window dùng để giữ cho bản đồ lấy robot làm trung tâm khi di chuyển xung quanh.
- transform_tolerance cấu hình cho khoảng thời gian trễ lớn nhất khi biến đổi (transform). planner_patiente cấu hình cho khoảng thời gian đợi lâu nhất tính bằng giây để module cố gắng tìm đường đi hợp lý trước khi thực hiện hủy thao tác đang thực hiện.
- width, height, resolution dùng để cấu hình cho các chiều và độ phân giải của costmap tính bằng mét.

Trong quá trình điều hướng, em sử dụng gói “gmcl” để tạo nên nút dành cho phần nội địa hóa. Nút này được thực hiện theo phương pháp GMCL (general Monte Carlo Localization) (<http://wiki.ros.org/gmcl>).

3.2 Triển khai hệ thống

3.2.1 Công cụ mô phỏng và mô hình hóa

Trong quá trình phát triển, chương trình “rviz” là công cụ được sử dụng để mô hình hóa 3D cho hoạt động của robot (<http://wiki.ros.org/rviz>). Với rviz, ta có thể kiểm soát được vị trí của robot trên bản đồ hoặc đặt một vị trí mục tiêu mà robot phải đi đến từ một nơi cách xa đó. Ngoài ra, chương trình “Gazebo” có thể được sử dụng cho mục đích mô phỏng robot và môi trường xung quanh để chạy các bài thử nghiệm trước khi thực hiện trong thực tế.

3.2.2 Điều hướng cho robot sử dụng ROS và 2D LiDAR

Trong cơ chế điều hướng mà em thực hiện, em sử dụng gói phần mềm “costmap_2d” với các lớp là lớp bản đồ tĩnh (static map layer), lớp vật cản (obstacle layer) và lớp đệm (inflation layer). Với lớp bản đồ tĩnh, em sử dụng node “map_server” để cung cấp một bản đồ cho lớp này. Dữ liệu quét của 2D LiDAR được sử dụng cho lớp vật cản. Đối với lớp đệm, ta có thể tối ưu hóa các tham số để thêm giá trị xung quanh các vật thể nhạy cảm (như một lớp đệm xung quanh các vật thể). Sơ đồ liên kết giữa các node trong phần mềm được thể hiện qua hình dưới đây:

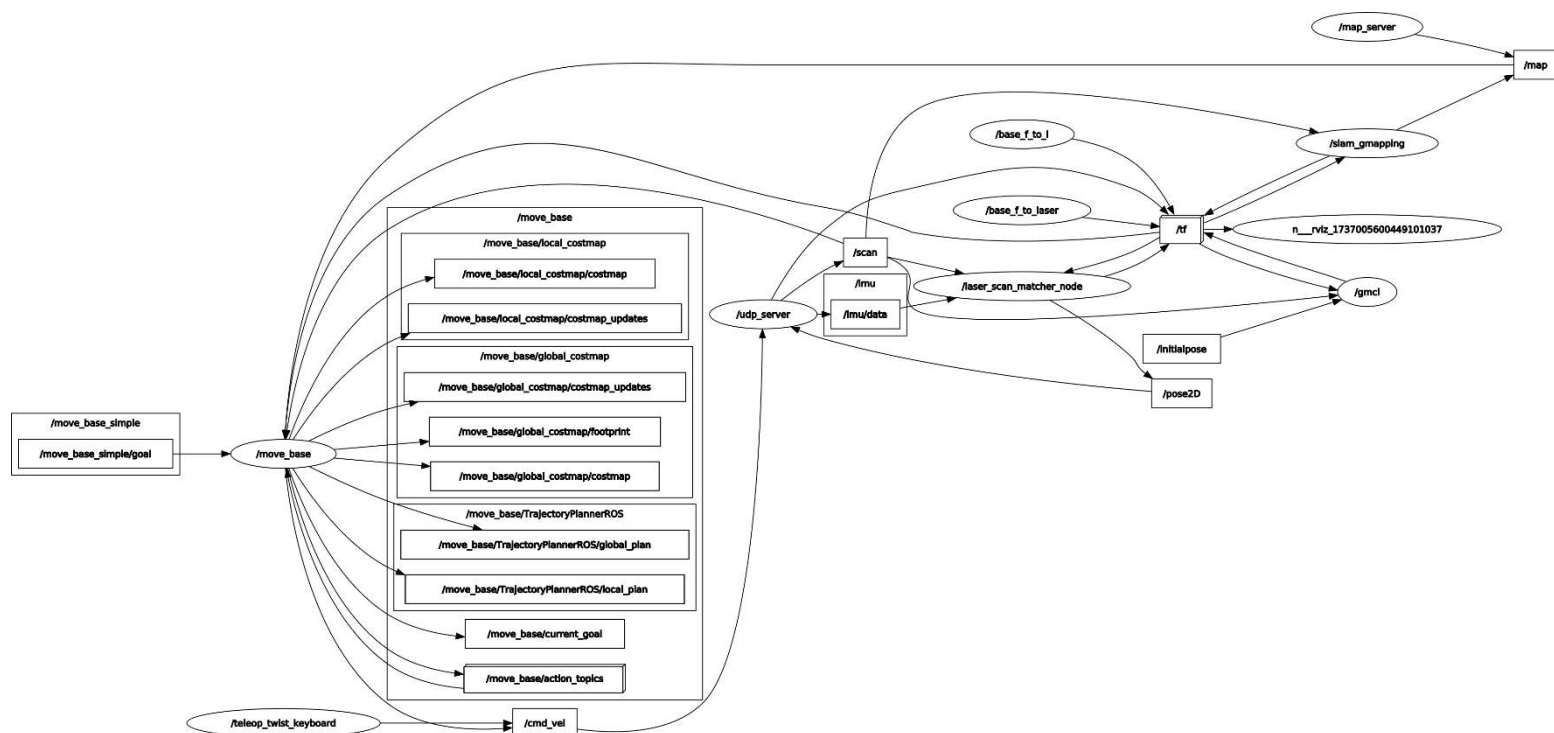


Figure 12. Sơ đồ quan hệ giữa các nút trong phần mềm

Như có thể thấy trên hình, hệ thống sử dụng thư viện “tf” để xuất bản (publish) tọa độ transform của các cảm biến và khung của robot qua nốt “base_f_to_laser”. Nốt này cung cấp thông tin về tọa độ tf giữa khung và LiDAR nằm trên robot cho các thành phần khác trong hệ thống. Quá trình SLAM sẽ được thực hiện bởi hai nốt “gmcl” và “slam_gmapping” có nguồn từ hai gói phần mềm ROS cùng tên. Nốt “gmcl” sẽ “đăng ký” các để nhận các gói tin từ các tin nhắn transform, các mẫu quét của LiDAR và sẽ xuất bản tin nhắn transform giữa map_frame và odometry_frame. Tin nhắn transform này thể hiện sự xê dịch do sai số khi robot tự ước tính vị trí của nó trong quá trình di chuyển. “gmcl” đảm nhiệm vai trò nội địa hóa (localization) trong quá trình SLAM của hệ thống. Cho quá trình xây dựng bản đồ, nốt “slam_gmapping” sẽ lấy đầu vào là tin nhắn về dữ liệu quét của cảm biến (*sensor_msgs/Scan*) được truyền không dây từ robot nhờ nốt “udp_server” và các tin nhắn transform cần thiết cho LiDAR, khung robot và odometry để cho đầu ra là bản đồ lưới chiếm dụng (Occupancy grid map).

“laser_scan_matcher_node” cho phép khớp các mẫu quét từ các tin nhắn *sensor_msgs/Scan* nhờ thuật toán ICP [10], và xuất bản vị trí ước tính của laser dưới dạng một transform. Phần mềm này có thể được sử dụng mà không cần thông tin odometry từ các cảm biến khác (ví dụ: encoder bánh xe) và có thể đóng vai trò là nốt ước tính vị trí – cung cấp odometry. Nốt này sẽ lấy đầu vào là tin nhắn mẫu quét được của LiDAR (*sensor_msgs/Scan*) và tin nhắn IMU (*sensor_msgs/IMU*)

(dùng để ước lượng sự thay đổi về góc định hướng của robot (yaw) trong quá trình di chuyển).

Trong module điều hướng của hệ thống, nốt “move_base” là một thành phần lớn gồm nhiều chương trình nhỏ. Nốt này được trợ giúp bởi nốt “gmcl” trong việc cục bộ hóa, kết hợp thuật toán tìm đường trên cả bản đồ (global planner) và thuật toán tìm đường cục bộ (local planner) để thực hiện quá trình định hướng cho robot. Khi nốt này nhận được mục tiêu trên bản đồ (qua rviz 2D hoặc qua terminal), nó bắt đầu xuất bản tin nhắn vận tốc điều khiển *cmd_vel* cho robot base controller.

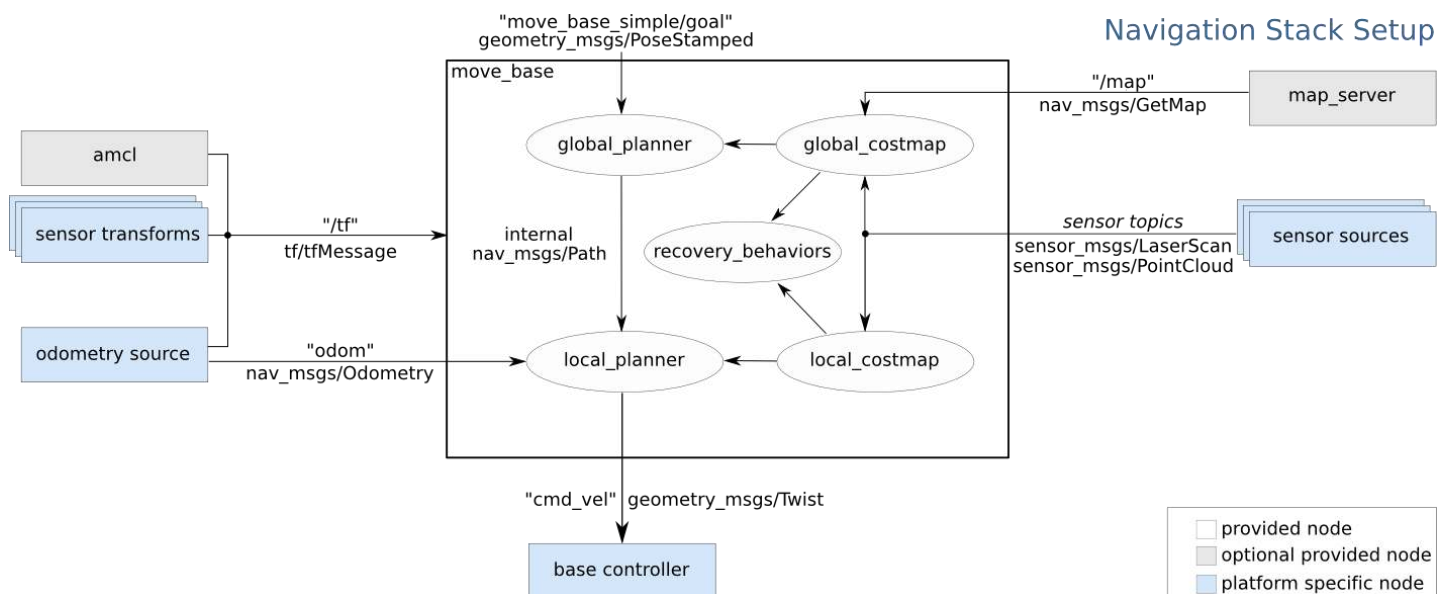


Figure 13. Sơ đồ góc nhìn từ trên cao cho navigation stack

Trên đây là một góc nhìn từ trên cao cho thấy rõ sự tương tác của “move_base” với các nốt khác trong hệ thống. “move_base” cung cấp một giao diện dành cho ROS để có thể cấu hình, chạy và tương tác với navigation stack của robot.

3.2.3 Hành vi phục hồi của robot

Khi robot bắt đầu hoạt động và nút “move_base” được cấu hình theo đúng các yêu cầu đề ra [], robot sẽ cố gắng điều hướng đến vị trí mục tiêu với một sai số có thể được chỉ định tùy chọn. Khi robot nhận thấy rằng nó bị mắc kẹt, “move_base” sẽ thực hiện các hành vi để phục hồi và tiếp tục hoạt động. Một cách mặc định, “move_base” sẽ thực hiện những hành động sau:

move_base Default Recovery Behaviors

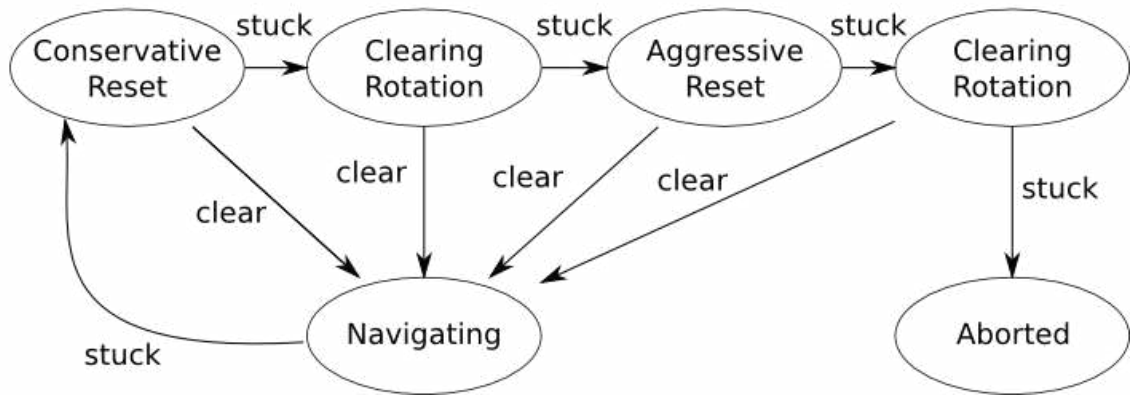


Figure 14. Quá trình phục hồi của robot

Đầu tiên, những vật cản nằm ngoài vùng được chỉ định sẵn sẽ được loại bỏ khỏi bản đồ của robot. Tiếp theo, robot sẽ cố gắng thực hiện tự quay vòng xung quanh tại chỗ để loại bỏ chướng ngại vật trong không gian xung quanh. Nếu hành động này vẫn thất bại, robot sẽ buộc phải thực hiện loại bỏ các chướng ngại vật xung quanh vùng không gian hình chữ nhật xung quanh nó để robot có thể thực hiện quay vòng tại chỗ. Sau đó, robot sẽ thực hiện quay vòng một lần nữa. Nếu tất cả mọi hành động đều thất bại, robot sẽ coi như vị trí mục tiêu đặt ra là không khả thi và sẽ báo cho người dùng là quá trình điều hướng đã thất bại.

CHƯƠNG 4. THỬ NGHIỆM VÀ KẾT QUẢ

4.1 Thử nghiệm

4.1.1 Thiết lập môi trường thử nghiệm

Trong thử nghiệm này, robot được chạy thử trong môi trường chỉ có vật thể có thể được quét bởi LiDAR 2D. Sau đây là hình ảnh môi trường trong thực tế:



Figure 15. Thiết lập cho thử nghiệm

4.1.2 SLAM

Các thử nghiệm cho hệ thống robot tự hành di động được chạy trên phần mềm rviz, như được thể hiện qua hình dưới đây. Đầu tiên là thử nghiệm chức năng thiết lập bản đồ đồng thời xác định vị trí (SLAM). Các nút cần thiết cho quá trình này được chạy từ file launch. Để thu thập dữ liệu cho quá trình SLAM, em sử dụng gói “teleop_twist_keyboard” để điều khiển robot từ xa bằng bàn phím. Bản đồ được thiết lập cũng như quá trình thu thập dữ liệu có thể được quan sát trong phần mềm rviz:

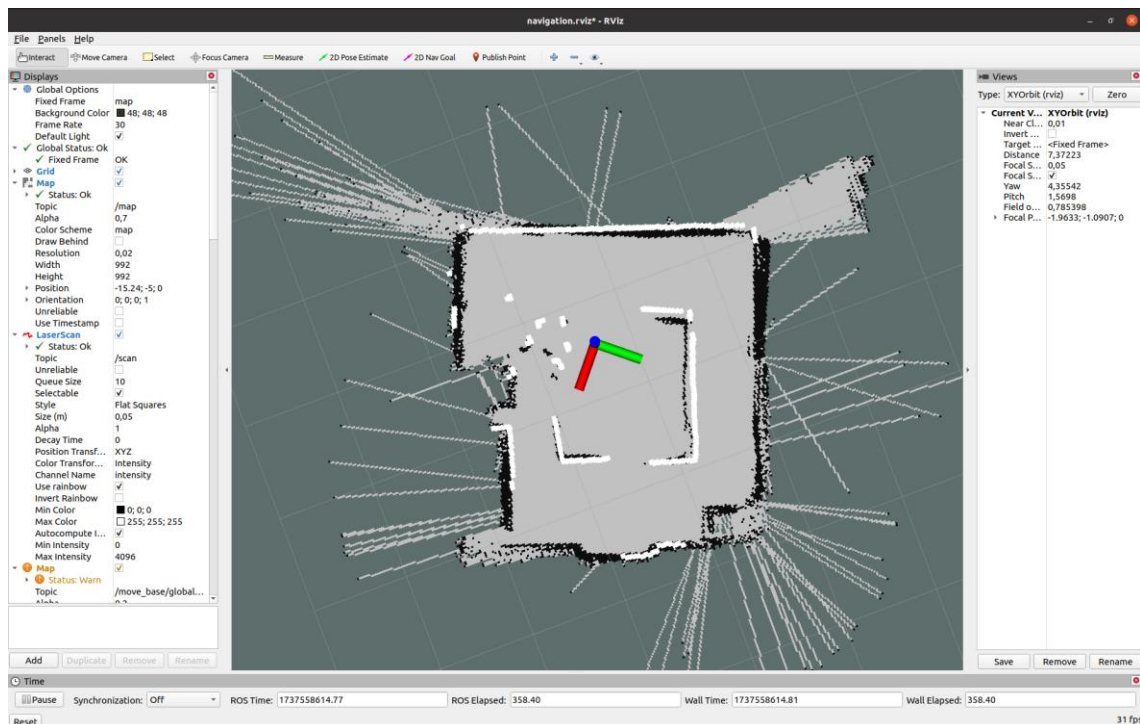
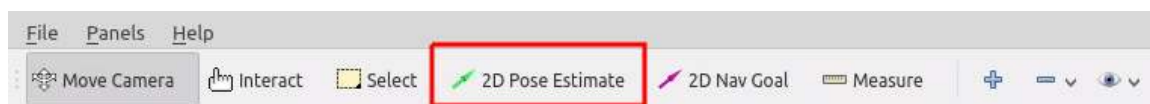


Figure 16. Bản đồ được thiết lập nhờ SLAM thể hiện trên rviz

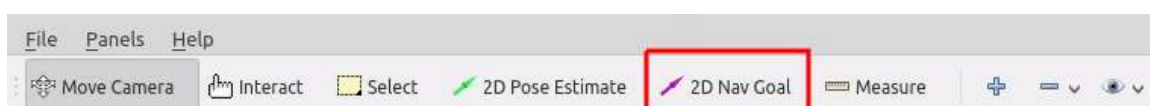
4.1.3 Điều hướng

Thí nghiệm được thực hiện trên phần mềm rviz để có thể quan sát quá trình điều hướng một cách trực quan. Nút “move_base” phụ trách việc thiết lập đường đi toàn bản đồ (global path planning) và thiết lập đường đi cục bộ (local path planning). Trong rviz, ta có thể đặt vị trí mục tiêu và cho robot tự động định hướng đến vị trí mục tiêu đó như hình 17.

Trong cửa sổ rviz, nút **2D Pose Estimate** có thể được sử dụng để ước tính vị trí và hướng của robot trên bản đồ (*estimated_robot_pose*):



Nút **2D Nav Goal** cho phép gửi một vị trí mục tiêu cho nút move_base bằng cách gửi một tin nhắn dạng *geometry_msgs/PoseStamped* thuộc topic *move_base_simple/goal*:



Các thử nghiệm đặt ra nhằm kiểm tra các chức năng liệt kê sau:

- Điều hướng: khả năng đi đến vị trí mục tiêu đề ra.
- Tránh vật cản: khả năng tránh né con người và vật cản.
- Phục hồi: khả năng tiếp tục di chuyển và định hướng sau khi bị dừng lại.

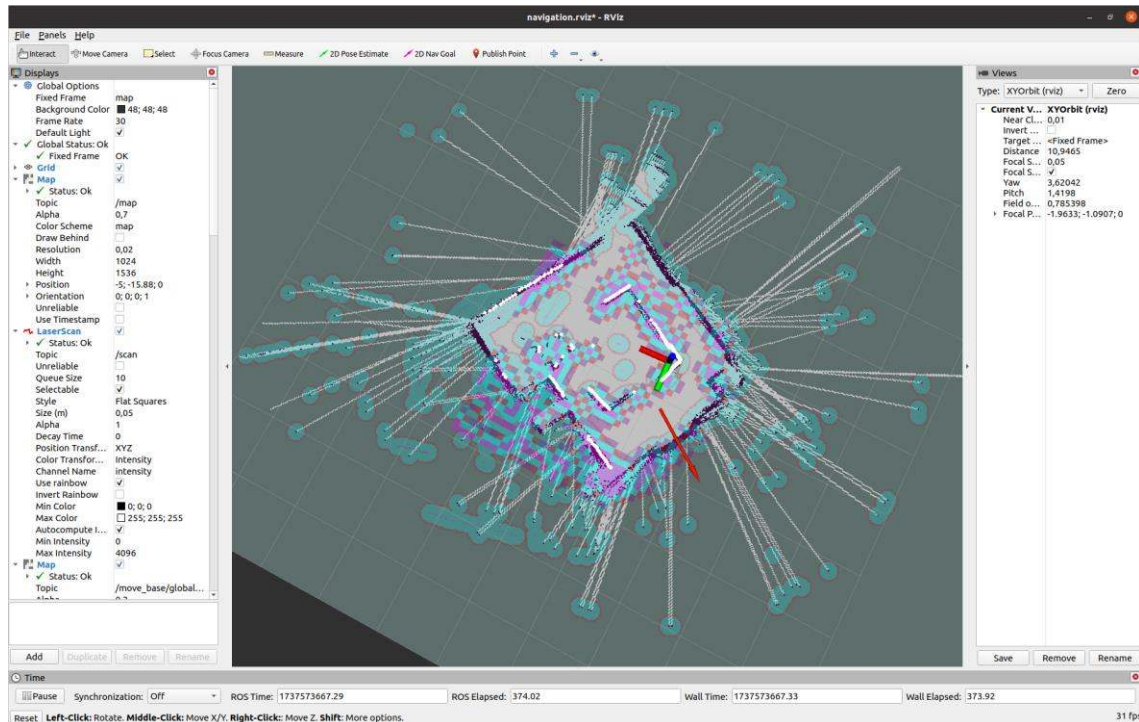


Figure 17. Điều hướng cho robot minh họa trên rviz.

4.1.4 Kết quả

Trong thí nghiệm này, với chức năng SLAM, kết quả cho thấy hệ thống có thể xây dựng bản đồ một cách khá chính xác với thực tế và không gặp nhiều sai số. Với chức năng định hướng, kết quả thể hiện robot có khả năng định hướng đến vị trí đặt ra và có thể tránh khỏi vật cản.

Tuy nhiên với chức năng định hướng, sự xê dịch trong quá trình ước lượng vị trí (odometry) còn lớn do giới hạn về khả năng xử lý của vi điều khiển cũng như độ trễ lớn trong quá trình giao tiếp không dây khiến cho khả năng điều hướng trở nên thiếu chính xác theo thời gian.

CHƯƠNG 5. KẾT LUẬN

KẾT LUẬN CHUNG

Đề tài này em đã trình bày một cách thực hiện xây dựng robot tự hành di động có khả năng thiết lập bản đồ và định hướng sử dụng LiDAR 2D cùng với ROS. Mặc dù đã thỏa mãn phần nào các chức năng đề ra, kết quả thực nghiệm cho thấy hệ thống vẫn còn các điểm yếu cần khắc phục. Hệ thống được thực hiện dựa trên stack ROS định hướng cơ bản, độ trễ trong mạng lưới giao tiếp không dây ảnh hưởng nhiều tới các nút phần mềm trong robot mỗi cách khác nhau. Khả năng tính toán từ xa của hệ thống cho chức năng điều hướng không đạt yêu cầu về độ chính xác cao do thông lượng cao và độ trễ đường truyền.

HƯỚNG PHÁT TRIỂN

Trong tương lai, để hệ thống trở nên hoàn thiện và áp dụng được cho nhiều điều kiện môi trường một cách chính xác và tin tưởng hơn, sau đây là một số định hướng:

- Cải thiện khả năng tối ưu băng thông trong truyền thông.
- Cải thiện khả năng kháng lại các ảnh hưởng do độ trễ đường truyền.
- Tăng cường tích hợp các cảm biến khác để khắc phục điểm yếu của LiDAR 2D, ví dụ như camera hoặc sử dụng 3D depth point cloud.
- Cải thiện các thuật toán cho SLAM và navigation bằng cách sử dụng các thuật toán mạnh hơn như *Cartographer* (từ Google) hoặc *RTAB-Map* (Real-Time Appearance-Based Mapping) đối với SLAM và *Hybrid A Planner* đối với chức năng định hướng.

TÀI LIỆU THAM KHẢO

- [1] M. Quigley, K. Conley, B. Gerkey, J. Faust, T. Foote, J. Leibs, R. Wheeler and A. Y. Ng, "vol 3," in *Ros: an open-source robot operating system*, ICRA workshop on open source software, 2009.
- [2] S. Zaman, W. Slany and G. Steinbauer, "ROS-based mapping, localization and autonomous navigation using a Pioneer 3-DX robot and their relevant issues," in *ROS-based mapping, localization and autonomous navigation using a Pioneer 3-DX robot and their relevant issues*, Communications and Photonics Conference, 2011, pp. 1-5.
- [3] K. T. A. I. J. T. a. T. O. Y. Ochiai, *Remote control system for multiple mobile robots using touch panel interface and autonomous mobility*, pp. 3272-3277, 2014 IEEE/RSJ International Conference on Intelligent Robots and Systems.
- [4] Foote and Tully, "tf: The transform library," pp. 1-6, 2013.
- [5] Y. Pyo, H. Cho, L. Jung and D. Lim, *ROS Robot Programming (Second Edition)*, Ruby Paper, 2017.
- [6] Khalil and M. A. Alshikh, "<https://wiki.ros.org/gmcl>," 2019. [Online].
- [7] E. J. Krakiwsky, C. B. Harris and R. V. C. Wong, "A Kalman filter for integrating dead reckoning, map matching and GPS positioning," *IEEE PLANS '88., Position Location and Navigation Symposium, Record*, pp. 39-46, 1988.
- [8] M. A. A. Khalil, "Development of a New Technique in ROS for Mobile Robots Localization in Known-Based 2D Environments," 2022.
- [9] L. Zhang, R. Zapata and P. Lepinay, "Self-adaptive monte carlo for single-robot and multi-robot localization," no. IEEE International Conference on Automation and Logistics, 2009.
- [10] A. Censi, "An ICP variant using a point-to-line metric," *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, 2008.
- [11] M. A. a. H. I. Alshikh Khalil, "Development of a New Technique in ROS for Mobile Robots Localization in Known-Based 2D Environments," *Tishreen University Journal of Research and Scientific Studies - Engineering Sciences Series*, no. Vol. 43 No. 6, pp. 119-137, 2021.
- [12] Khalil, Alshikh and M. Ali, "gmcl As a Proposed Replacement to amcl in ROS for Mobile Robots Localization in Known-Based 2D Environments," p. 11, 2021.